Journal of Cloud Computing

RESEARCH Open Access

MkRP: multiple k registry placement for fast container deployment on edge computing

Chunggeon Song[1†], Heonchang Yu[1*†] and Joon-Min Gil[2*†]

Abstract
Edge computing reduces the response time of real-time services with handling dynamic traffic reliably. Edge servers with limited resources often utilize container technology, which provides a lightweight execution environment. When deploying containers on edge servers, a container image is required and it is predominantly downloaded from a remote registry. Therefore, these operations are heavily influenced by network overhead between the container deployment system and the remote registry. Through motivation experiments specifically designed to identify these network overheads, we demonstrate that container pulling time increases in proportion to the physical distance between two hosts and varies flexibly based on runtime conditions. In this paper, we define a system model to place multiple registries for high-speed container deployment and propose a technique for clustering edge servers and selecting leaders within each cluster based on the affinity between edge servers and network overhead. We also propose a method for selectively deploying registries based on the idle resources of edge servers. We validate the proposed technique through simulation experiments. The results show consistent performance improvements regardless of the edge server count or the k value.

Keywords Container deployment, Registry placement, Edge computing

Introduction

Edge computing is a distributed computing paradigm in which servers located in close proximity to end-users perform tasks traditionally handled by centralized cloud infrastructures. This model seeks to reduce service latency and conserve network bandwidth [1]. The proliferation of Internet of Things (IoT) devices has significantly increased the volume of valuable data generated at the edge. Moreover, the rise of latency-sensitive applications such as augmented reality, autonomous vehicles, healthcare, and disaster response further underscores the critical role of edge computing [2]. In particular, healthcare systems leverage edge resources to process real-time patient data, enabling rapid interventions in emergency scenarios [3, 4].

In addition to healthcare, edge computing has recently been adopted in the field of federated learning to optimize network bandwidth usage between geographically distributed clients and central servers [5, 6]. In this context, containerization technologies provide a portable and efficient solution for deploying aggregators on edge servers. Additionally, the deployment speed of containers can be significantly improved by leveraging registry optimization techniques.

[†]Chunggeon Song, Heonchang Yu and Joon-Min Gil contributed equally to this work.

*Correspondence:
Heonchang Yu
yuhc@korea.ac.kr
Joon-Min Gil
jmgil@jejunu.ac.kr
[1]Department of Computer Science and Engineering, Korea University, 145 Anam-ro Seongbuk-gu, Seoul 02841, Republic of Korea
[2]Department of Computer Engineering, Jeju National University, 102 Jejudaehak-ro, Jeju 63243, Jeju Special Self-Governing Province, Republic of Korea

Multiple edge servers at the end of a network for edge computing have relatively limited resources compared to the central cloud, and require technologies that can reliably handle the fluid traffic generated by device mobility. To meet these requirements for edge servers, container virtualization, which provides a lightweight execution environment, and cloud-native technologies to run scalable services are being leveraged and are considered de facto standards.

Each edge server has a service scope of a certain size, running some tasks of real-time services for clients within its scope, or performing preprocessing on data generated by sensor devices located nearby. In addition, to provide multi-tenancy on edge servers, container operating platforms will utilize very different types of container images in short period of time. In particular, when executing a task based on Function as a Service(FaaS) on an edge server, as the size of the work unit decreases, both the variety of containers and the quantity needed to execute the function increase.

If the container image required for container deployment is not available locally, image pulling is performed to download the container image from a remote registry. However, due to the nature of edge computing, edge servers are geographically distributed over long distances and are connected to a Wide Area Network(WAN). Accordingly, network overhead has a significant impact on container deployment speed. As a result, research on container placement techniques that consider network overhead has emerged [7–15].

Knob et al. [7] proposed a technique for modeling the network topology between edge servers in the form of a weighted graph, clustering it based on the fluid communities algorithm, and distributing a registry for each cluster. Temp et al. [8] proposed a strategy to deploy a container registry considering user mobility. Roges et al. [9] proposed a policy that dynamically provisions the registry according to the storage of the server and the needs of the application. Because these traditional techniques only perform one policy or one clustering technique, there is a low probability of finding container images in registries deployed nearby. Additionally, it has the disadvantage of inconsistent container deployment time because it does not take into account dynamic changes in network traffic.

In this paper, we propose a technique for dividing edge servers into k groups and deploying registry servers to improve the deployment speed of containers operating in a system performing edge computing. The proposed technique performs two-stage registry placement based on network overhead and affinity-based similarity of the container images used. It also covers techniques for distributing container registries based on idle resource information and container usage frequency within each cluster. The contributions of this paper are as follows.

- We present a new model that expresses the network overhead that occurs when deploying containers on multiple geographically distributed edge servers and the affinity between edge servers.
- We present a multiple k registry placement technique that selects several leaders based on affinity and network overhead. It then strategically deploys registries while considering the idle resources of each edge server.
- Through simulation, we confirmed that the proposed algorithm shows stable performance under various numbers of edge servers and number of clusters k.

The rest of this paper is as follows. Background section provides the background of the research, and Proposed solution section explains the proposed k registry placement technique for fast container deployment. Then Evaluation section describes simulation-based experiments to demonstrate the performance of MkRP. Related work section describes related work on container deployment and registry placement. Finally, Conclusion section concludes the paper and outlines future research plans.

Background

We redefined and modeled the registry deployment problem to improve container deployment speed in a distributed system environment performing edge computing. In this section, we describe types of network connectivity along with the system architecture of the edges commonly cited when defining registry server deployment problems, and describe a motivation experiment to confirm the validity of the research.

Multiple regionally separated edge servers have connections of wired internet and wireless network, and data communication between two edge servers goes through many router hops. In communication through a WAN network, the data transmission and reception path can be determined depending on the routing algorithm, and since all network lines have limited bandwidth, the data movement speed can be affected by traffic. Figure 1 conceptually represents various types of network connection relationships between two edge servers.

Both wireless and wired communication are possible between the edge servers, e_i and e_j. The wireless connection is connected directly to the overlapping communication range of the Base Station, and the wired connection is connected through four router hops. Additionally, wireless communication is not possible between

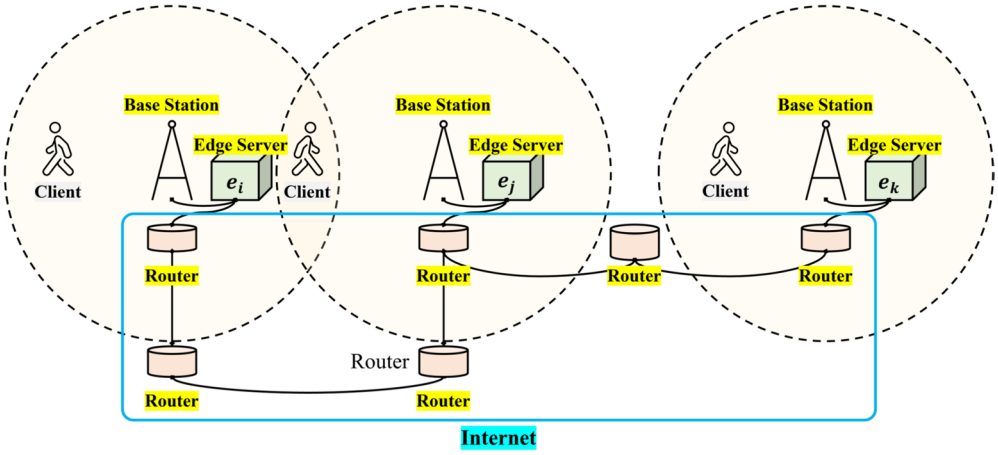

Fig. 1 Types of network connections between two edge servers

edge servers e_j and e_k, and only wired communication via 3 router hops is possible. As a result, wireless communication is not possible between edge servers over some distance, and wired communication is used using the Internet. In particular, communication between the edge server and remote registry server, such as container image pulling, is performed only through the Internet communication with dynamic traffic. This communication method between edge servers makes it difficult to model network overhead, and makes network traffic fluid over time due to various factors.

To provide empirical motivation for our proposed approach, we conducted a preliminary experiment to quantify the network overhead incurred by wired communication between geographically dispersed edge servers. Specifically, we deployed multiple container registry servers across different countries using the Azure public cloud platform, thereby varying the physical distance between the client and registry locations.

The container image pulling speed was measured from a client located within Korea University. Three registry locations were considered: co-located within the same country as the client (South Korea), in a nearby country (Japan), and on a different continent (United States). All registry servers were instantiated using identical virtual machine templates on Azure. The experiment utilized Docker version 24.0.4 as the container runtime, and the base image was Ubuntu 20.04.

Each image pull was repeated five times, and the average download time was computed to minimize variability. The results, presented in Fig. 2, illustrate the impact of physical distance on container image pull latency.

Experimental result shows the image pulling time is increased in proportion to the physical distance between the client and the registry server. Through these experimental results, it was confirmed that when pulling container images from a registry server located in a short distance, the container deployment time can be reduced. Then, to check the liquidity of network overhead generated during container pulling, an experiment was conducted to measure the Round Trip Time(RTT) between the client and the registry server at 1 second intervals. Figure 3 shows the results of measuring the RTT between the client and three registry servers for 12 hours in the same environment as the first motivation experiment.

Experimental result shows some overall difference in RTT proportional to the physical distance and great liquidity of RTT over time. In particular, between 4 o'clock and 5 o'clock, the RTT for the kr registry server was higher than that for the jp registry server regardless of the physical distance due to the impact on network traffic. These results confirmed that network overhead changes over time, and that periodic network overhead monitoring is necessary to improve container deployment.

Proposed solution

System model

This section describes various system models for quantifying the speed of image pulling involved in container deployment on an edge server with limited resources. In particular, it defines the components and metrics that affect the image pulling time of multiple edge servers connected by a network over a WAN. For the sake of simplicity, we assume that each pair of edge servers is connected by reliable channels. Table 1 describes various symbols used for defining different basic system components. Figure 4 represents a structural model that describes the relationships between system components.

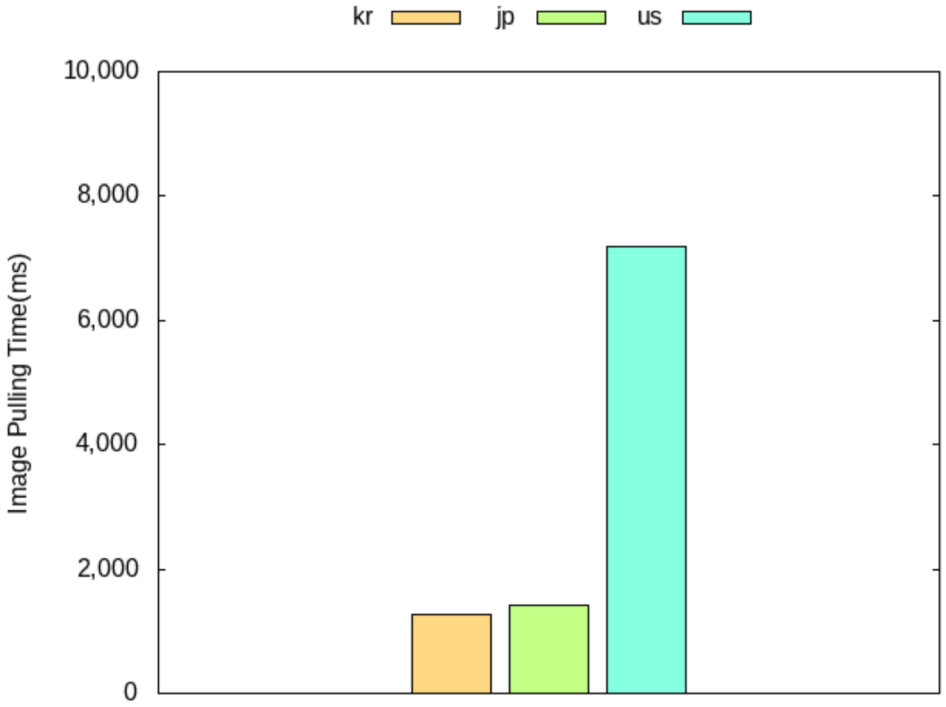

Fig. 2 Image pulling time depending on physical distance

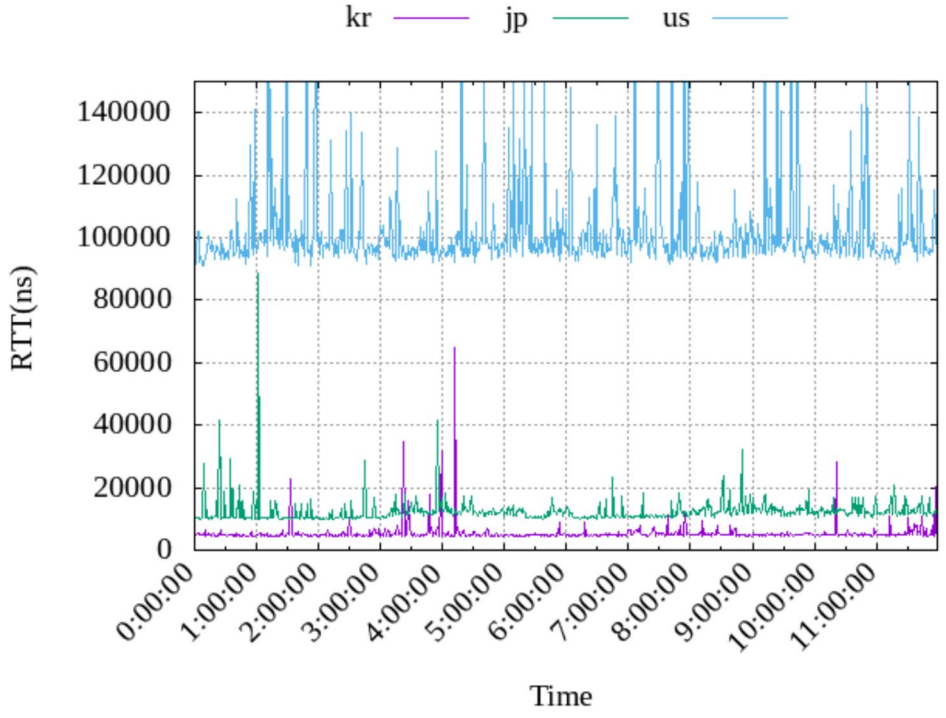

Fig. 3 RTT changes over time

Table 1 Notations

Symbol	Description
E	$\{e_i \mid e_i$ is ith edge server $1 \leq i \leq N\}$
R	$\{r_j \mid r_j$ is jth registry $1 \leq j \leq K\}$
$S_{i,j}^{RTT}$	One or more of RTTs occurred between e_i and e_j Collected during Time Interval M
S^{RTT}	Set of $S_{i,j}^{RTT} 1 \leq i, j \leq N$
S_i^{image}	Set of Sample Data for Image Request in e_i Collected during Time Interval M
S^{image}	Set of $S_i^{image} 1 \leq i \leq N$
$I(e_i)$	The ratio of idle resources for e_i
$O(e_i e_j)$	Network Overhead between e_i and e_j
$A(e_i e_j)$	Affinity between e_i and e_j
λ	Threshold value for idle resources required to perform the role of a leader

The local profiles of edge servers exchange RTT messages with each other to generate profile data, which is then transmitted to the central cloud server at intervals of L cycles. The Global Profiler of the central cloud server utilizes the collected profile data to generate G^O and G^A. Finally, the Registry Deployment Manager performs the MkRP technique at intervals of $M(M > L)$ cycles and deploys multiple registries. We explain three equations that define various metrics are explained.

$$O(e_i e_j) = \frac{\sum_{i=1}^{|S_{i,j}^{RTT}|} a_i}{2|S_{i,j}^{RTT}|}, a_i \in S_{i,j}^{RTT} \quad (1)$$

Equation (1) represents the formula for calculating the network overhead between edge servers e_i and e_j $O(e_i e_j)$. $S_{i,j}^{RTT}$ is an unfixed size RTT data set. This requires $N!$ calculations per M cycles. The average of the $S_{i,j}^{RTT}$ collected through message communication between e_i and e_j is calculated and divided by 2 to obtain the one-way communication time for sending and receiving RTT messages.

$$I(e_i) = 1 - (\text{cpuUsage}_i * \alpha + \text{diskUsage}_i * \beta) \quad (2)$$

Equation (2) represents the formula for calculating the ratio of idle resource for e_i. An environment in which all edge servers are homogeneous is assumed and cpuUsage_i and diskUsage_i used in $I(e_i)$ have values between 0 and 1. These metrics can be collected using Linux cgroups. α and β are weight values that control which resource type is given more weight when calculating the ratio of idle resources for each edge server.

$$A(e_i e_j) = \frac{S_i^{image} \cap S_j^{image}}{S_i^{image} \cup S_j^{image}} \quad (3)$$

Equation (3) represents the formula for calculating the affinity between edge servers e_i and e_j. This value is calculated as the ratio of the number of commonly pooled image sets to the total number of image sets requested from edge servers e_i and e_j.

MkRP

The multiple k registry placement algorithm, which is the core technique of the proposed MkRP, divides the edge servers in the E set into k groups and deploys multiple registry servers for each edge group and is expressed in Algorithm 1. Clustering is performed only when k is greater than $N/2$. This algorithm utilizes G^O which

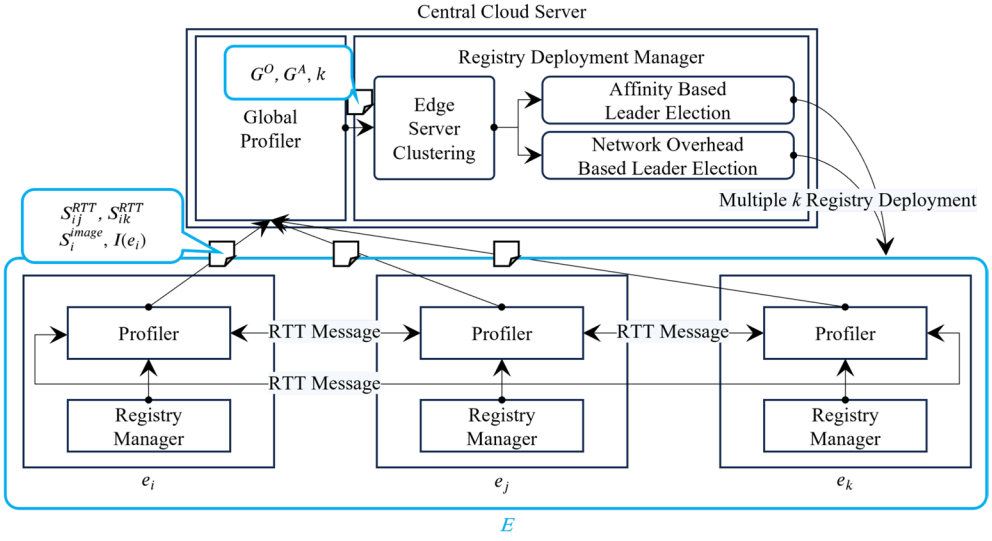

Fig. 4 System architecture

defines $O(e_i e_j)$ as a complete graph and G^A which defines $A(e_i e_j)$ as a complete graph between all edge servers every M cycles.

Require: E, G^O, G^A, k
1: $leader \leftarrow null$
2: /* first k registry placement */
3: $\{G_i\}_{i \in I} \leftarrow ClusterGraph(E, G^O, k)$
4: **for** G_i in $\{G_i\}_{i \in I}$ **do**
5: $leader \leftarrow ElectLeader(E, G_i, 'networkOverhead')$
6: **if** $leader \neq null$ **then** $DeploymentRegistry(leader)$
7: **end if**
8: **end for**
9: /* second k registry placement */
10: $\{G_i\}_{i \in I} \leftarrow ClusterGraph(E, G^A, k)$
11: **for** G_i in $\{G_i\}_{i \in I}$ **do**
12: $leader \leftarrow ElectLeader(E, G_i, 'affinity')$
13: **if** $leader \neq null$ **then** $DeploymentRegistry(leader)$
14: **end if**
15: **end for**

Algorithm 1 Multiple k registry placement algorithm

`ClusterGraph()` divides the complete graph received as an argument into *k* subgroups based on the index number of the edge server. `ClusterGraph()` can invoke a variety of algorithms capable of partitioning a complete graph into k communities. For this study we selected Fluid Communities (FluidC), which has been empirically shown to deliver the best performance for the edge-server clustering problem [16, 17]. FluidC is a propagation-based community-detection algorithm that models communities as fluids that expand and contract within the graph, much like opposing liquids filling a container. It achieves lower computational cost than traditional modularity-maximisation methods while yielding more stable partitions than label-propagation approaches.

`DeploymentRegistry()` deployments the registry service to the edge server. Container images stored in the registry are preferentially selected based on the recently used container image log data from the Leader Edge server. Then, the list of container image IDs recently retrieved from edge servers in the cluster is retrieved and additional container images are stored.

In Algorithm 1, line 3 expresses the operation of clustering the set of edge server into k subgroups. Loop in lines 4–8 expresses the feature of deployment the registry server by electing a leader among the edge servers in the subgroup based on network overhead. The loop in lines 10–14 performs the function of electing a leader among edge servers and distributing a registry server based on the complete affinity graph. The `ElectLeader()` function in lines 5 and 12 is expressed in detail in Algorithm 2.

Require: E, G, $graphType$
Ensure: $leader$
1: $leader \leftarrow e_1$
2: $minSum \leftarrow \sum_{Path \in G.Paths} Path.weight$
3: $maxSum \leftarrow 0$
4: $sum \leftarrow 0$
5: **for** v_i in G **do**
6: $sum \leftarrow 0$
7: **for** $Path$ in $G.Paths$ **do**
8: **if** $Path$ is connected to v_i **then**
9: $sum \leftarrow sum + Path.weight$
10: **end if**
11: **end for**
12: **if** $I(e_i) \geq \lambda$ **then**
13: **if** $graphType =' networkOverhead'$ **then**
14: **if** $minSum > sum$ **then**
15: $leader \leftarrow e_i$
16: $minSum \leftarrow sum$
17: **end if**
18: **else if** $graphType =' affinity'$ **then**
19: **if** $maxSum < sum$ **then**
20: $leader \leftarrow e_i$
21: $maxSum \leftarrow sum$
22: **end if**
23: **else**
24: $leader \leftarrow null$
25: **end if**
26: **else**
27: $leader \leftarrow null$
28: **end if**
29: **end for**

Algorithm 2 Leader election algorithm for ElectLeader()

In Algorithm 2, lines 5–29 perform an operation to find the leader by traversing the graph nodes. Lines 12–28 perform the function of electing a leader by using the sum of the weights between a specific node and the remaining nodes. If the input graphType value is 'networkOverhead' according to lines 13–17, the node with the minimum weight sum is elected as the leader. If the graphType value is 'affinity', the node with the maximum weight sum is elected as the leader according to lines 18–22. In line 12, if $I(e_i)$ is greater than the threshold value for idle resources λ, it is elected as the leader. As a result, if there is no edge server that satisfies the condition, MkRP does not perform deployment of regisrty.

Time complexity of MkRP

To explain the time complexity of the MkRP algorithm, we analyzed the time complexity of `ClusterGraph()` and `ElectLeader()` included in MkRP, respectively, and also explain the overall time complexity of the MkRP algorithm.

Because the physical order of edge servers does not change after initial deployment, the result of `ClusterGraph()`

is always the same. The time complexity of this function is $O(N)$. `ElectLeader()` is a complete graph composed of about N/k nodes because the G used as input elects the leader after clustering all edge servers into k numbers. And because the algorithm operates with a double for loop, it has $O((N/k)^2)$ time complexity. The MkRP algorithm overall consists of two iterations for registry placement based on network overhead and registry placement based on affinity. Accordingly, each has a complexity of $O(N/k)$. This algorithm includes the `ClusterGraph()` function and `ElectLeader()`, so the final algorithm complexity is as follows.

$$((N/k)^2) * (N/k) + N = O(N^3) \tag{4}$$

Since the efficiency of the algorithm increases proportionally to the input k, it is recommended to set the number of k to the maximum without running out of resources on the edge server.

Evaluation

Simulator

We developed a simulator in Go and Python language to verify the performance of M*k*RP, the proposed registry placement technique. The network modeling and clustering were implemented in Python using the NetworkX library, while the container image pulling logic was written in Go[].

Each edge server has a cache of a certain size, and when pulling images, container images stored in the cache are used preferentially. In addition, one or more registry server information can be registered on the edge server, and when deploying containers, container images are sequentially searched according to the registration order.

Every registry stores a certain number of container images, and container images are identified by ID. In the simulation, the option for the number of edge servers is set manually, and which container to deploy is determined using a weight-based random value. Container image pulling time is calculated based on the network overhead between the container deployment system and the registry that holds the requesting container image. Clustering for both G^O and G^A is performed via the FludiC-based `ClusterGraph()` function.

Figure 5 shows an example of a complete graph representing the network overhead generated in a certain resource management cycle when the number of edge servers is set to 5 and the simulation is performed. Figure 5a illustrates the overall graph topology, whereas Fig. 5b and c depict the graph partitioned into two distinct clusters. In this figure, the blue node is the leader of the subgraph and performs the role of a registry. The weight value of a graph edge is a measure of the network overhead between two edge servers. Among the nodes in each subgroup, the node with the minimum sum of $O(e_i, e_j)$ between the remaining modes is selected as the leader.

Figure 6 shows a complete graph expressing the affinity created in a certain resource management cycle in the same environment. Figure 6a represents the entire graph, and Fig. 6b and c represent graphs clustered into two subgraphs. In this figure, the red node is the leader of the subgraph and performs the role of a registry. Among the nodes in each subgroup, the node with the maximum sum of $A(e_i, e_j)$ between the remaining modes is selected as the leader. The weight value of a graph edge is a measure of the affinity between two edge servers.

Affinity values are utilized to elect a leader within each clustered edge-server group. During this affinity-based election, container images to be cached in the local registry are selected from the container-pulling logs recorded by all edge servers in the cluster. Consequently, the elected leader maintains on its local storage the images most frequently shared across its cluster, thereby enhancing the cache-hit ratio, reducing the average pulling distance, and ultimately accelerating container deployment.

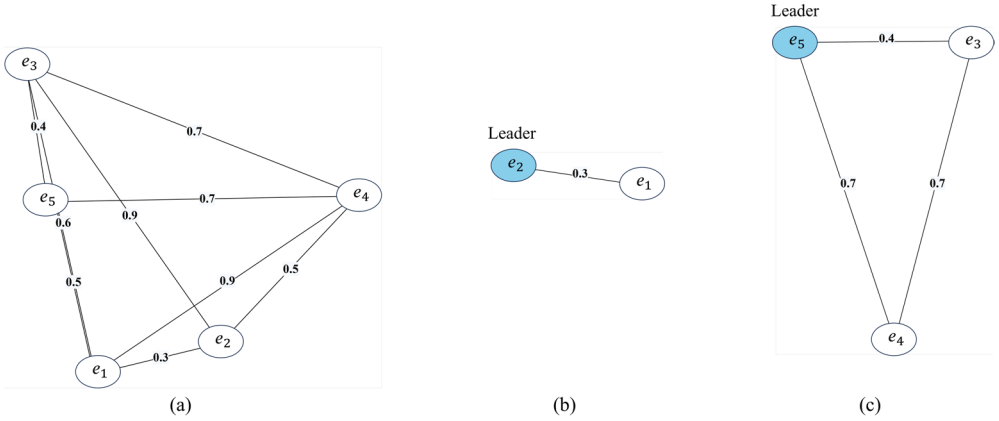

Fig. 5 Complete graph representing network overhead between edge servers: (a) Network overhead graph, (b) First subgraph, and (c) Second subgraph

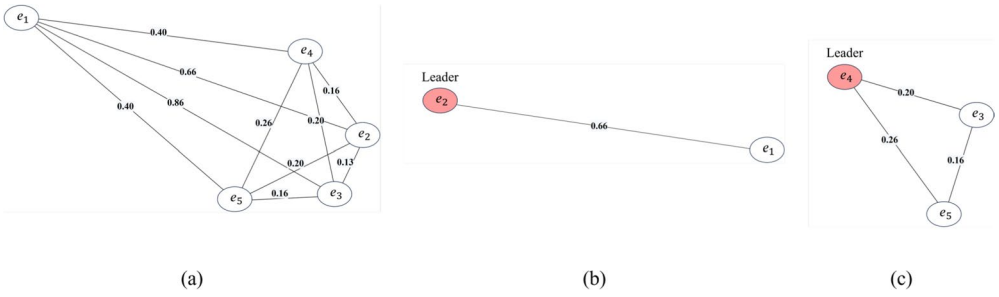

Fig. 6 Complete graph representing affinity between edge servers: (a) Affinity graph, (b) First subgraph, and (c) Second subgraph

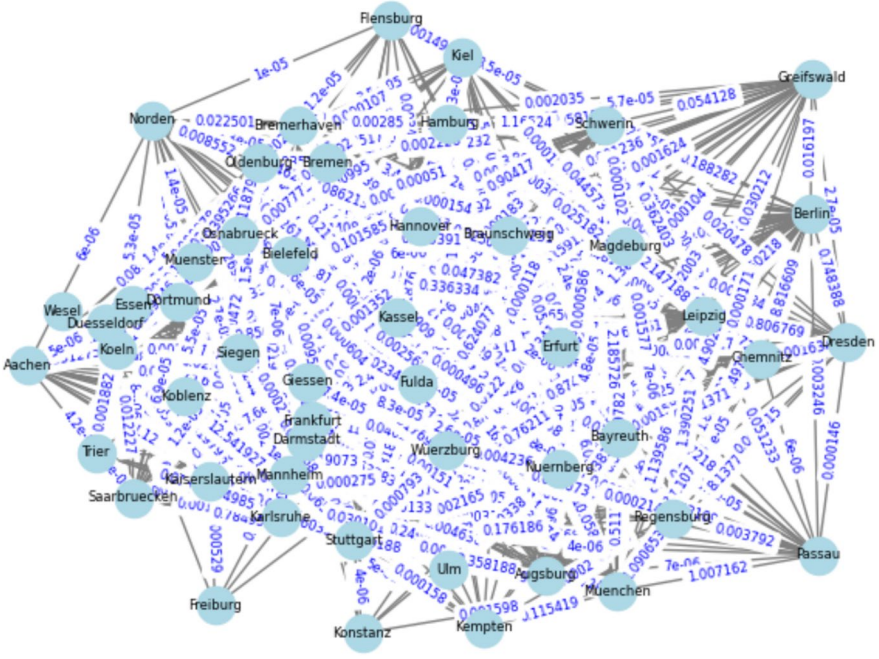

Fig. 7 germany50 network

To capture the temporal dynamics observed in real-world networks, we modeled the graphs using the dynamic traffic traces provided by SNDlib [18]. Figure 7 shows the model derived from the germany50 network data used in our experiments.

Simulation scenario

In the simulation experiment to verify the performance of M*k*RP, the number of registry servers was set to 5, and each registry server stores 100 container images. Straesser et al. [19] shows that the sizes of commonly used container images are very different. Accordingly, the size of container image was set to randomly generated size ranging from 1MB to 1000MB considering sampled container images of the Docker Hub container image. And each edge server has a cache size of 5GB. If the cache capacity is exceeded, the edge server replaces the newly downloaded container image with a less frequently used container. Container images requested from each edge server were requested from a specific registry server three times more frequently than the remaining registry servers. All edge servers request a container image once per second.

The experimental system for the simulation was an IBM System x3500 M4 server, equipped with an Intel Xeon E5-2650 CPU, 62 GB of memory, and 1 TB of storage. CentOS 7, running the Linux 4.15 kernel, was used as the operating system for the experiment.

Container images subject to pulling are stored in three types of spaces: 1) remote registry server 2) leader node within the edge group 3) local cache on the edge server. A remote registry server was modeled based on motivation experiment data, and the pulling time was calculated as Equation (6). When retrieving a container image from a remote registry connected to the network, the transfer time was calculated according to Equation (5). img_h^{size} is the size of the container image calculated in bits. e_i is an edge server that pulls container images, and r_j is a registry that provides container images. B^{max} represents the

maximum network bandwidth between e_i and r_j, and $B_{i,j}^{traffic}$ represents the traffic occurring on the network between e_i and r_j.

$$T^{transfer}(img_h^{size}, e_i, r_j) = img_i^{size}/B^{max} - B_{i,j}^{traffic} \quad (5)$$

$$T^{pulling} = \begin{cases} T^{transfer}(img_h^{size}, e_i, reg.) & \text{if the image is in remote registry} \\ T^{transfer}(img_h^{size}, e_i, leader) & \text{if the image is in leader} \\ 0 & \text{if the image is in local} \end{cases} \quad (6)$$

Container images are searched in the following order: local cache, G^O and G^A based two registries deployed on the edge server that plays the leader role, and five remote registries. There are four baselines: a case where MkRP is not used (non), a case where only MkRP's Network Overhead-based k registry placement technique is performed (mkrp no), a case where the algorithm from existing SOTA research [7] was implemented (comm) and both MkRP's Network Overhead-based k registry placement technique and Affinity-based k registry placement technique it is performed (mkrp no+af). An experiment was conducted to visualize the simulation operation process. In the experiment, container images were requested 10,000 times from 5 edge servers and resource management was performed every 1,000 seconds, and the results are shown in Fig. 8.

As the resource management technique is applied after 16 minutes, the result shows that the number of references to the container image in the remote registry gradually decreases. Depending on the preferences of clients using edge servers, specific container images were frequently pulled to edge servers. By utilizing container images through a registry built on a nearby edge server, the image pulling time was drastically reduced. Next, we describe different numbers of edge servers and *k* values and describe an experiment to check the average image pulling time.

Image pulling performance according to numbers of edge servers

We conducted an experiment to check the container image pulling performance of M*k*RP according to changes in the number of edge servers. The image pulling time was measured by setting the *k* value to 3 and changing the number of edge servers to 10, 20, 30, 40, and 50. In the experiment, resource management was performed at 15 minute intervals for 24 hours, and the results are shown in Fig. 9.

In the experimental results, 'mkrp no+af' showed 37.3%, 17.2% and 18.4% higher performance than 'non-mkrp', 'mkrp no' and 'comm' for all edge server numbers.

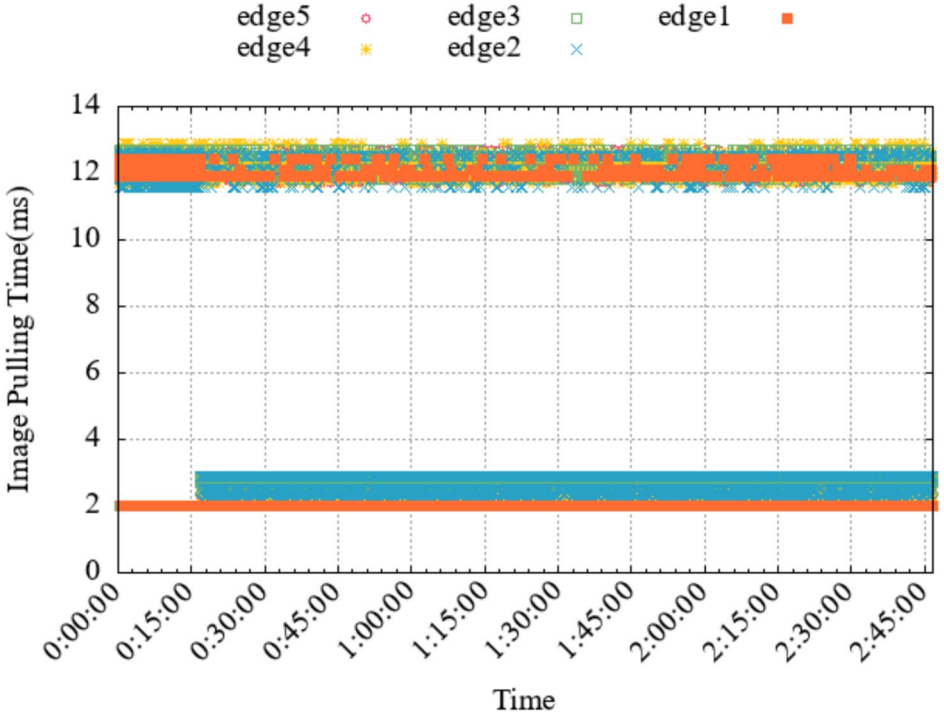

Fig. 8 Image pulling simulation

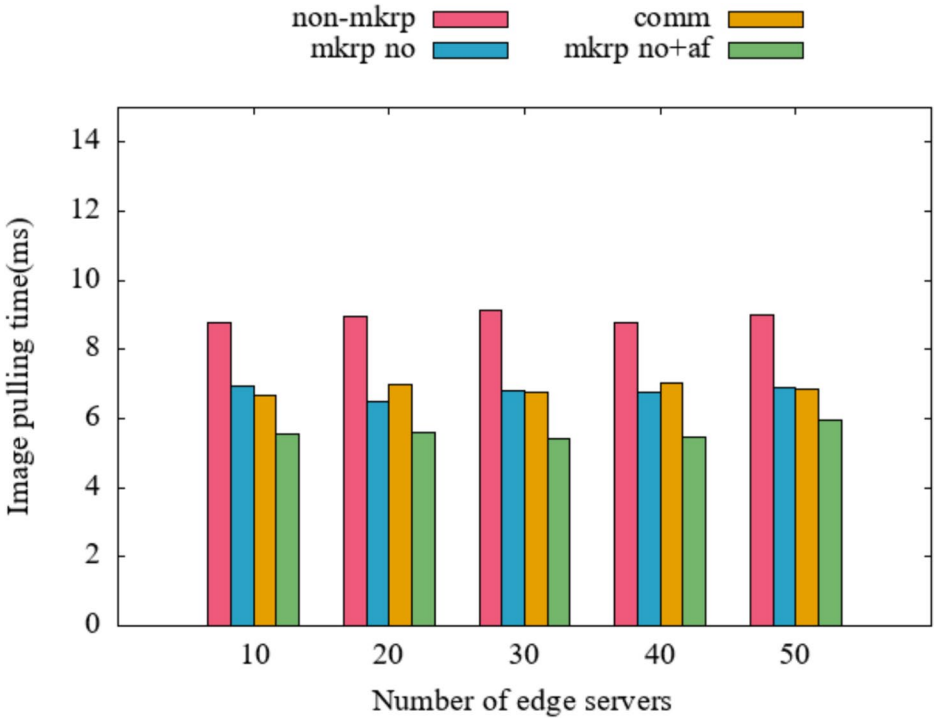

Fig. 9 Image pulling time according to the numbers of edge servers

In cases 'non-mkrp' and 'mkrp no', the average image pulling time decreased in proportion to the number of edge servers, and the reason for this phenomenon was that the hit rate increased as the frequency of utilizing the local cache of the edge server increased. The case 'mkrp no+af' showed consistent performance regardless of the number of containers. 'comm' is not suitable for a complete graph designed to model dynamic network conditions, resulting in lower performance when the number of edge servers is small.

It can be observed that 'mkrp no+af' shows significantly higher performance improvement compared to 'mkrp no'. Additionally, the performance difference between 'non-mkrp' and 'mkrp no' is smaller compared to the performance difference between 'mkrp no+af'. The reason for this performance improvement is that the service usage patterns among users of edge computing-based cloud services are similar, resulting in higher affinity. Consequently, the number of times container images are pulled by registries deployed on leader nodes within each cluster increases. If the network traffic fluidity within clusters divided into k parts increases, the performance improvement of registry placement based on network overhead is likely to grow further.

Image pulling performance according to various k values

We conducted an experiment to check the container image pulling performance of M*k*RP according to changes in *k*, the number of subgroups. In the experiment, the number of edge servers was set to 50 and the image pulling time was measured while changing the *k* value to 5, 10, and 15. M*k*RP was operated at 15 minute intervals for 24 hours, and the results are shown in Fig. 10.

In the experimental results, 'mkrp no+af' showed 36.7%, 19.3%and 36.6% higher performance than 'non-mkrp', 'mkrp no' and 'comm' for all *k*. However, performance does not increase when the *k* value exceeds a certain value. As the value of *k* increases, the number of edge groups increases and the number of registry servers deployed increases, and the rate of referencing remote registries decreases, which is the cause of performance improvement. However, if *k* increases above a certain number, registry operation overhead increases more than the level of performance improvement, so it is important to set an optimized *k*. 'comm' exhibited exponentially decreasing performance as the value of k increased. Consequently, it has a limitation in that it cannot flexibly adjust the value of k.

Related work

Research on container deployment

Existing research on container deployment is evolving with more than one purpose. Lin et al. [20] proposed a technique to deploy containers for the purpose of energy consumption while satisfying the SLA. Hu et al. [21] proposed a technique to deploy containers by considering

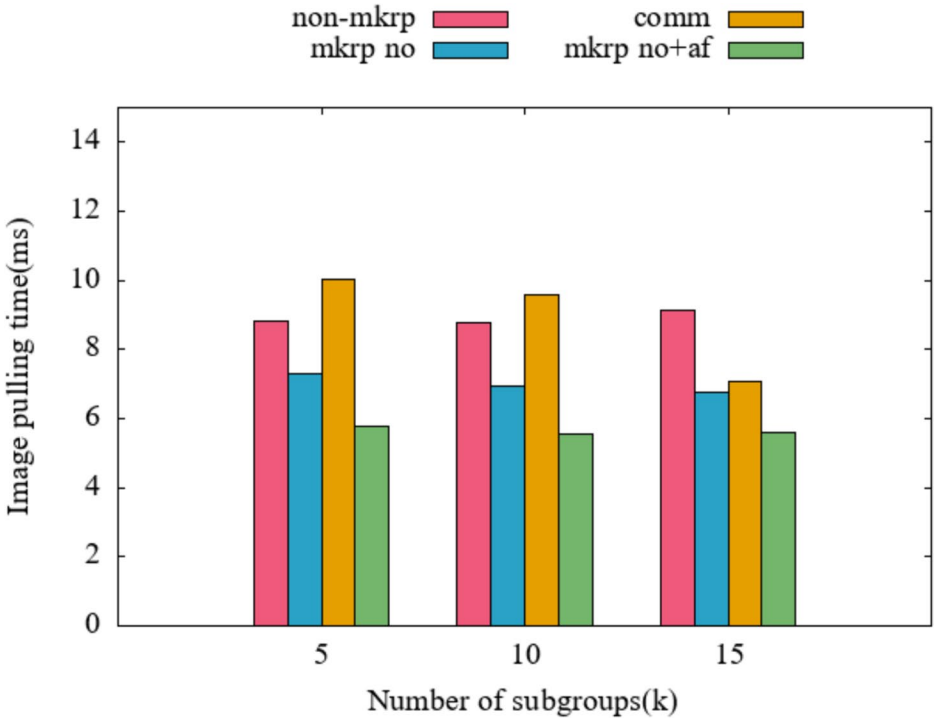

Fig. 10 Image pulling time for different *k* values

load balancing and dependencies between containers while ensuring multi-resource requirements. In order to perform tasks sensitive to response speed on edge servers, it is important to quickly deploy a lightweight execution environment [22]. Accordingly, various studies have emerged to improve container deployment speed. Harter et al. [23] proposed a technique to improve container deployment speed by utilizing a centralized storage server shared between the container deployment system and the container registry.

Research on container registry placement

Various existing studies have investigated efficient techniques for dividing edge servers into several subgroups and distributing one container registry to each sub-group. Knob et al. [7] modeled the static network topology as a complete graph and developed a clustering technique. Temp et al. [8] proposed a technique for deploying a container registry based on user mobility. However, in situations where network traffic is elastic, these studies show that it is difficult for the complete graph to store accurate network overhead information, and thus the performance improvement rate of the average container deployment time is reduced. Roges et al. [9] proposed a technique for dynamically deploying the registry according to the server's storage size and application requirements. This utilizes dynamic data, but has limitations in selecting a registry leader by utilizing metrics for one type of resource and traffic. The M*k*RP is differentiated from existing studies in that it utilizes a combination of network overhead and affinity and increases the probability of finding a container image in a registry distributed nearby.

Conclusion

The demand for using containers on edge computing will increase, and the types of images for deploying containers will also become more diverse. In particular, when providing multi-tenancy in cloud services, multiple tenant users frequently deploy various types of services, and it is impossible to provide stable services with a single container registry. For this reason, the importance of additional registry deployment within edge servers will increase.

In this paper, we proposed a technique for dividing edge servers into *k* groups and deploying registry servers to improve the deployment speed of containers operating in a system performing edge computing. A simulation experiment was conducted to verify the performance of the proposed technique and showed that dramatic performance improvement was observed regardless of the number of edge servers and the *k* value. This study is the first to address the registry placement problem by incorporating affinity between edge servers and to propose a system model and algorithm that deploys registries based on multiple independent metrics.

Cloud service providers can reduce container deployment times by applying MkRP to Kubernetes-based FaaS or serverless. In particular, it is expected that user

satisfaction will increase while guaranteeing SLO for real-time services based on edge computing. In the future, we plan to develop a technique to hierarchically cluster multiple edge servers, elect a leader, and deploy a registry.

Acknowledgement
This research was supported by the Regional Innovation System & Education (RISE) program through the Jeju RISE center, funded by the Ministry of Education (MOE) and the Jeju Special Self-Governing Province, Republic of Korea (2025-RISE-17-001) .

Authors' contributions
All the authors contributed equally to work. All authors have read and agreed to the published version of the manuscript.

Funding
Not applicable.

Data availability
No datasets were generated or analysed during the current study.

Declarations

Ethics approval and consent to participate
Not applicable.

Consent for publication
Not applicable.

Competing interests
The authors declare no competing interests.

Received: 10 August 2024 / Accepted: 29 September 2025
Published online: 03 November 2025

References

1. Bonomi F, Milito R, Zhu J, Addepalli S (2012) Fog computing and its role in the internet of things. In: Proceedings of the first edition of the MCC workshop on Mobile cloud computing (MCC), August 17, 2012, Helsinki, Finland. ACM, New York, pp 13-16
2. Cao K, Liu Y, Meng G, Sun Q (2020) An overview on edge computing research. IEEE Access 8:85714–85728
3. Oueida S, Kotb Y, Aloqaily M, Jararweh Y, Baker T (2018) An edge computing based smart healthcare framework for resource management. Sensors 18:4307
4. Ray PP, Dash D, De D (2019) Edge computing for internet of things: a survey, e-healthcare case study and future direction. J Netw Comput Appl 140:1–22
5. Wehbi O, Arisdakessian S, Wahab OA (2023) Fedmint: intelligent bilateral client selection in federated learning with newcomer iot devices. IEEE Internet Things J 10(23):20884–20898
6. Bai J, Chen Y (2023) The node selection strategy for federated learning in UAV-assisted edge computing environment. IEEE Internet Things J 10(15):13908–13919
7. Knob LA, Faticanti F, Ferreto T, Siracusa D (2021) Community-based placement of registries to speed up application deployment on edge computing. In: Proceedings of the IEEE International Conference on Cloud Engineering (IC2E), October 4-8, 2021, Virtual. IEEE, New York, pp 147-153
8. Temp DC, Souza PSSD, Lorenzon AF, Luizelli MC (2023) Mobility-aware registry migration for containerized applications on edge computing infrastructures. J Netw Comput Appl 217:103676
9. Roges L, Ferreto T (2023) Dynamic provisioning of container registries in edge computing infrastructures. In: Proceedings of the 24th Brazilian Symposium on High Performance Computing Systems (WSCAD), October 17-20, 2023, Porto Alegre, Brazil. IEEE, New York, pp 85-96
10. Nathan S, Ghosh R, Mukherjee T, Narayanan K (2017) Comicon: A co-operative management system for docker container images. In: Proceedings of the IEEE International Conference on Cloud Engineering (IC2E), April 4-7, 2017, Vancouver, Canada. IEEE, New York, pp 116-126
11. Faticanti F, Pellegrini FD, Siracusa D, Santoro D, Cretti S (2019) Cutting throughput with the edge: App-aware placement in fog computing. In: Proceedings of the IEEE International Conference on Edge Computing and Scalable Cloud (EdgeCom), June 21-23, 2019, Paris, France. IEEE, New York, pp 196-203
12. Kangjin W, Yong Y, Hanmei L, Ma L (2017) FID: A faster image distribution system for docker platform. In: Proceedings of the IEEE International Workshops on Foundations and Applications of Self* Systems (FAS*W), September 18-22, 2017, Tucson, AZ. IEEE, New York, pp 191-198
13. Littley M, Anwar A, Fayyaz H, Fayyaz Z, Tarasov V, Rupprecht L, Skourtis D, Mohamed M, Ludwig H, Cheng Y, Butt AR (2019) Bolt: Towards a scalable docker registry via hyperconvergence. In: Proceedings of the IEEE International Conference on Cloud Computing (CLOUD), July 8-13, 2019, Milan, Italy. IEEE, New York, pp 358-366
14. Ahmed A, Pierre G (2018) Docker container deployment in fog computing infrastructures. In: Proceedings of the IEEE International Conference on Edge Computing (EDGE), July 2-7, 2018, San Francisco, CA. IEEE, New York, pp 1-8
15. Rossi F, Cardellini V, Presti FL, Nardelli M (2020) Geo-distributed efficient deployment of containers with kubernetes. Comput Commun 159:161–174
16. Parés F, Garcia-Gasulla D, Vilalta A, Moreno J, Ayguadé E, Labarta J, Cortés U, Suzumura T (2018) Fluid communities: A competitive, scalable and diverse community detection algorithm. The Sixth International Conference on Complex Networks and Their Applications. Springer, Cham, pp 229–240
17. Nathan, S., et al. (2017). Comicon: A co-operative management system for Docker container images. 2017 IEEE International Conference on Cloud Engineering (IC2E). IEEE, 2017, pp 116–126
18. SNDlib Team (2025) SNDlib dynamic traffic dataset: germany50 (v1.0) [Dataset]. Zuse Institute Berlin. https://sndlib.zib.de. Accessed 20 June 2025
19. Straesser M, Bauer A, Leppich R, Herbst N, Chard K, Foster I, Kounev S (2023) An empirical study of container image configurations and their impact on start times. In: Proceedings of the IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid), May 1-4, 2023, Bangalore, India. IEEE, New York, pp 94-105
20. Lin C, Chen J, Liu P, Wu J (2018) Energy-efficient core allocation and deployment for container-based virtualization. In: Proceedings of the IEEE International Conference on Parallel and Distributed Systems (ICPADS), December 11-13, 2018, Singapore. IEEE, New York, pp 93-101
21. Hu Y, Laat DC, Zhao Z (2019) Multi-objective container deployment on heterogeneous clusters. In: Proceedings of the IEEE International Symposium on Cluster, Cloud and Grid Computing (CCGrid), May 14-17, 2019, Larnaca, Cyprus. IEEE, New York, pp 592-599
22. Darrous J, Lambert T, Ibrahim S (2019) On the importance of container image placement for service provisioning in the edge. In: Proceedings of the IEEE International Conference on Computer Communication and Networks (ICCCN), July 29-August 1, 2019, Valencia, Spain. IEEE, New York, pp 1-9
23. Harter T, Salmon B, Liu R, Arpaci-Dusseau AC, Arpaci-Dusseau RH (2016) Slacker: fast distribution with lazy docker containers. In: Proceedings of the USENIX Conference on File and Storage Technologies (FAST 16), February 22-25, 2016, Santa Clara, CA. USENIX Association, Berkeley, pp 181-195

Publisher's Note